

Arquitectura de computadores: Programación Vectorial en Ensamblador x86-64 (NASM/Linux)

J.Perez, S.Rojas

2 de octubre de 2026

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
2. Descripción del entorno de pruebas	3
2.1. Hardware	3
2.2. Software	4
3. Diseño y arquitectura de los programas	4
4. Implementación escalar	5
4.1. <code>sum_array</code>	5
4.2. <code>compute_stats</code>	6
4.3. <code>normalize_array</code>	7
5. Implementación vectorial (AVX2)	8
5.1. Elección de AVX2 frente a SSE y AVX-512	8
5.2. <code>sum_array</code> : patrón base reutilizado	8
5.3. <code>compute_stats</code> : media, varianza y mínimo/máximo	9
5.4. Estrategia de reducción y precisión numérica	10
5.5. Versiones originales y evolución del código	11
5.6. Manejo del remanente (<i>tail handling</i>)	11
5.7. <code>normalize_array</code>	11
6. Casos de prueba	12
7. Resultados de rendimiento	14
7.1. Tiempos de ejecución	14
7.2. Speedup vs. tamaño de N	14
7.3. Resultados de <code>perf stat</code>	15
8. Evidencia de la sesión de depuración con GDB	16
8.1. Acumuladores de <code>sum_array</code> durante el bucle vectorial	16
8.2. Reducción horizontal en <code>double</code>	17
8.3. Varianza acumulada en <code>double</code>	18
8.4. Estado de <code>ymm0</code> y <code>ymm1</code> en <code>normalize_array</code>	19
8.5. Memoria del arreglo de salida	19
8.6. Comparación de iteraciones: escalar vs. vectorial	19

8.7. Manejo del remanente: prueba con $N = 15$	21
9. Conclusiones	22
9.1. Limitaciones observadas	23
9.2. Trabajo futuro	23
A. Diagramas de arquitectura	24

1. Introducción

Este informe documenta el desarrollo del proyecto “Normalizador estadístico vectorizado”, cuyo objetivo es implementar y comparar dos versiones funcionalmente equivalentes de un mismo conjunto de kernels de cómputo sobre arreglos de números en punto flotante de precisión simple: una versión escalar, escrita en NASM usando únicamente instrucciones que procesan un elemento a la vez, y una versión vectorizada, escrita en NASM usando las extensiones AVX2, que procesa ocho elementos por iteración mediante registros YMM de 256 bits. Ambas versiones calculan la suma, la media, la varianza poblacional, el mínimo y el máximo de un arreglo de entrada, y generan su versión normalizada mediante la puntuación estándar: $y[i] = (x[i] - \mu)/\sigma$. El propósito del ejercicio es comparar el modelo de ejecución SIMD (*Single Instruction, Multiple Data*) con el modelo escalar tradicional (SISD), aplicando los requisitos de alineación de memoria y el manejo de los elementos restantes cuando el tamaño del arreglo no es múltiplo del ancho vectorial. Además, se evalúa cuantitativamente el rendimiento de ambas implementaciones mediante el tiempo de ejecución, la aceleración (*speedup*) y las instrucciones por ciclo (IPC).

1.1. Objetivos del proyecto

- Implementar correctamente los tres kernels (`sum_array`, `compute_stats` y `normalize_array`) en ambas variantes.
- Verificar su correctud contra una referencia en NumPy, incluyendo los casos borde obligatorios.
- Medir el tiempo de ejecución y el speedup logrado por la vectorización en distintos tamaños de entrada.
- Documentar, mediante GDB, el contenido de los registros vectoriales y de memoria durante la ejecución del kernel vectorizado.

2. Descripción del entorno de pruebas

2.1. Hardware

Las pruebas se corrieron sobre un procesador Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz, dentro de una máquina virtual Linux sobre Windows (WSL2, host DESKTOP-NHU7N79). La salida de `"lscpu — grep -E 'Model name—Flags'"` confirma el modelo y la presencia de la bandera `avx2`:

```

1  Model name:   Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz
2  Flags:        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
3                pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm
4                arch_perfmon rep_good nopl xtopology cpuid tsc_known_freq pni
5                pclmulqdq ssse3 fma cx16 pdcn pcid sse4_1 sse4_2 movbe popcnt aes
6                xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch pti ssbd
7                ibrs ibpb stibp fsgsbase bmi1 avx2 smep bmi2 erms invpcid rdseed adx
8                smap clflushopt xsaveopt xsavec xgetbv1 xsaves md_clear flush_l1d
9                arch_capabilities

```

Listing 1: Salida de `lscpu` confirmando soporte AVX2

Para la sección de resultados de rendimiento es relevante además el tamaño de las cachés de este procesador, obtenido con `lscpu | grep -i cache`: L1d de 32 KiB y L2 de 256 KiB por núcleo, y L3 compartida de 8 MiB (4 núcleos físicos / 8 hilos).

2.2. Software

Tabla 1: Versiones de las herramientas utilizadas

Herramienta	Versión
NASM	2.16.01
GCC	13.3.0 (Ubuntu 13.3.0-6ubuntu 24.04.1)
GDB	15.0.50.20240403-git (Ubuntu 15.0.50.20240403-0ubuntu1)
Sistema Operativo	Ubuntu 24.04.3 LTS (noble), sobre WSL2

La sesión de depuración con GDB de la Sección 8 se realizó en otro equipo (AMD Ryzen 5 5600H, Ubuntu 26.04 LTS, GDB 17.1, NASM 3.01 y GCC 15.2.0), por lo que las versiones de la Tabla 1 no corresponden a esa sesión.

Los programas se ensamblaron con NASM [1] (sintaxis Intel) y se enlazaron con GCC sobre Linux.

3. Diseño y arquitectura de los programas

El diseño de ambas versiones se documentó en cuatro diagramas, entregados como archivo independiente (`docs/Diagramas.pdf`) y disponibles también en línea¹. Las páginas completas se incluyen en el Apéndice A (Figuras 2 a 5). Los diagramas corresponden a las versiones finales de los kernels: acumuladores en `double` para la suma y la varianza, y reducción horizontal en `double`.

- **Diagrama 1, arquitectura de software (Figura 2):** muestra el *driver* en C (`driver.c`, idéntico para ambas versiones) y las dos librerías de ensamblador (`stats_scalar.asm` y `stats_vector.asm`, representadas como `libstats_scalar` y `libstats_vector`), que exportan los mismos símbolos. El *driver* lee el archivo de entrada, reserva `in` y `out` con `aligned_alloc(32, ...)`, llama una vez a `sum_array` para reportar la suma, repite `compute_stats` y `normalize_array` el número de veces indicado, cronometrando cada llamada con `clock_gettime`, y escribe la salida y el resumen de estadísticos. El Makefile enlaza una sola librería por ejecutable. El diagrama recoge además los argumentos de cada función según la ABI System V AMD64 (`sum_array: rdi=arr, esi=n; compute_stats: rdi=arr, esi=n, rdx=mean*, rcx=var*, r8=min*, r9=max*; normalize_array: rdi=in, rsi=out, edx=n, xmm0=mean, xmm1=stddev`), el retorno de la suma en `xmm0`, los registros que debe preservar el *callee* (`rbx, rbp, r12--r15`), los que pueden destruirse en una llamada (el resto, incluidos `xmm0--xmm15`) y la alineación de `rsp` a 16 bytes antes de cada `call`. Como `compute_stats` llama internamente a `sum_array`, respalda antes sus punteros en registros *callee-saved*.
- **Diagrama 2, flujo de control (Figura 3):** compara el bucle escalar (un elemento por vuelta, acumulando en `double` con `cvtss2sd` y `addsd`, sin cola) con el vectorial de `sum_array`: inicialización de los dos acumuladores `double` (`ymm0` para los elementos 0–3 y `ymm2` para los 4–7), cálculo del límite vectorial con `and ecx, ~7`, bucle de bloques de 8 (dos `vcvtss2pd`, dos `vaddpd` y `add eax, 8`), reducción horizontal en `double` (`vaddpd, vextractf128, vaddpd` y `vhaddpd`), cola escalar de $n \bmod 8$ elementos (`vcvtss2sd` y `vaddsd`) y conversión final a `float` antes del `ret`. Para $n = 20$ el bucle vectorial da 2 vueltas (16 elementos) y la cola procesa los 4 restantes. El diagrama incluye también las tres fases de `compute_stats` (media mediante `sum_array` y `vdivss`; varianza en `double` con `vsubpd, vmulpd` y `vaddpd`, la misma reducción y `vdivsd` seguido de `vcvtss2sd`; mínimo y máximo en `float` con `vminps/vmaxps` y reducción con `vextractf128` y `vs_hufps`), con la rama de $n = 0$ que escribe 0,0 en las cuatro salidas, y los dos caminos de `normalize_array`: retorno inmediato si $n \leq 0$ y, en caso contrario, la comprobación de $\sigma = 0$ (que evita la división por cero)

¹Diagramas.pdf (SharePoint)

o la normalización con `vsubps` y `vdivps` sobre ocho carriles. Cada fase vectorial de ocho elementos va seguida de su cola escalar, con el mismo esquema que `sum_array`.

- **Diagrama 3, asignación de registros (Figura 4):** seis tablas, una por etapa del kernel vectorial, que indican qué contiene cada registro: (1) entrada a `compute_stats`; (2) respaldo de punteros en `rbx` y `r12--r15` y alineación de la pila antes de llamar a `sum_array`; (3) media, con `ymm0/ymm2` como acumuladores y `ymm1/ymm3` como bloques convertidos; (4) varianza, con `ymm0/ymm4` como acumuladores y la media replicada en `ymm1`; (5) mínimo y máximo en `ymm0` y `ymm1`; y (6) `normalize_array`, con la media y la desviación replicadas en `ymm0` y `ymm1`. Una séptima tabla resume los mismos roles en la versión escalar. En las etapas 3 y 4 cada `ymm` se interpreta como cuatro valores de 64 bits (`double`); en las etapas 5 y 6, como ocho valores de 32 bits (`float`).
- **Diagrama 4, memoria (Figura 5):** disposición de `in[]` y `out[]` para $n = 20$. Los 80 B de datos se completan con 16 B de relleno hasta 96 B, múltiplo de 32, y la base de ambos arreglos está alineada a 32 B por `aligned_alloc`. La versión escalar avanza 4 B por iteración (20 iteraciones). La vectorial avanza 32 B por iteración: el mínimo, el máximo y la normalización usan una carga `vmovaps` de 32 B (alineada, desplazamientos 0 y 32), mientras que la suma y la varianza usan dos `vcvtps2pd` de 16 B por semibloque (desplazamientos 0, 16, 32 y 48), que no exigen alineación. En ambos casos la cola procesa los 4 elementos restantes (desplazamientos 64, 68, 72 y 76), para un total de $2 + 4 = 6$ iteraciones. El diagrama indica también que el arreglo de entrada se recorre cinco veces (suma del *driver*, media, varianza, mínimo y máximo, y normalización), que el relleno nunca se lee y que $N = 0$ no llega a esta etapa, porque el *driver* lo rechaza antes de reservar memoria.

Estos diagramas sirven de referencia para las Secciones 4 y 5.

4. Implementación escalar

La implementación escalar, ubicada en `asm/scalar/stats_scalar.asm`, procesa un elemento del arreglo por iteración. Los datos de entrada y los resultados de las funciones se representan en precisión simple (`float32`); sin embargo, la suma y la varianza utilizan precisión doble (`float64`) para los cálculos intermedios. Esta decisión reduce la pérdida de precisión durante la acumulación, especialmente en arreglos grandes y en sumas con cancelación.

Esta es la versión final del kernel escalar. La versión original, que acumulaba en `float` y no incluía la conversión a `double`, se conserva sin modificar en el repositorio y se describe en la Sección 5.5.

Las operaciones sobre valores individuales emplean instrucciones escalares SSE/SSE2, como `cvtsd2ss`, `addsd`, `subsd`, `mulsd`, `divsd`, `movss` y `comiss`. El uso de registros XMM no implica que el cálculo sea vectorial: estas instrucciones operan únicamente sobre el elemento situado en la parte baja del registro.

Las tres funciones respetan la convención System V AMD64 ABI. Los argumentos enteros y los punteros se reciben en `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`, mientras que los argumentos de punto flotante se reciben en registros XMM. La función `sum_array` devuelve su resultado en `xmm0`. Cuando se utilizan registros *callee-saved*, se preservan sus valores y se restauran antes de retornar.

4.1. `sum_array`

Recorre el arreglo con un acumulador en `double` (`xmm0`): cada elemento se convierte de `float` a `double` con `cvtsd2ss` antes de sumarse con `addsd`. Al terminar el recorrido, el resultado se convierte de vuelta a `float` con `cvtsd2ss`, ya que la firma de la función devuelve `float` en `xmm0`.

```

1 sum_array:
2     xor     eax, eax
3     xorpd   xmm0, xmm0           ; acumulador EN DOUBLE = 0.0
4 .sum_loop:
5     cmp     eax, esi

```

```

6     jge     .sum_done
7     cvtss2sd xmm1, [rdi + rax*4]    ; arr[i]: float -> double
8     addsd   xmm0, xmm1              ; acumulador += arr[i], en double
9     inc     eax
10    jmp     .sum_loop
11  .sum_done:
12    cvtsd2ss xmm0, xmm0              ; convierto el resultado final a float
13    ret

```

Listing 2: sum_array (versión escalar): acumulación en double

4.2. compute_stats

Para calcular la media se reutiliza `sum_array` mediante “call `sum_array`”. Como esta llamada destruye los registros callee-saved, los punteros de salida (mean, var, min, max), el puntero al arreglo y `n` se guardan primero en los registros callee-saved `rbx`, `r12`, `r13`, `r14` y `r15` (`n` se guarda además en la pila, ya que no quedan más registros callee-saved disponibles, cuidando alinear la pila a 16 bytes antes de la llamada). Como `sum_array` ya devuelve la suma convertida a `float`, la división entre `n` se hace en `float` con `divss`, igual que antes del cambio:

```

1     mov     rbx, rdx                ; mean*
2     mov     r12, rcx                ; var*
3     mov     r13, r8                 ; min*
4     mov     r14, r9                 ; max*
5     mov     r15, rdi                ; arr
6     push    rsi                     ; guardar n (doble push para alinear a 16 bytes)
7     push    rsi
8     call    sum_array                ; xmm0 = suma(arr), ya convertida a float
9     pop     rsi
10    pop     rsi
11    cvtsi2ss xmm1, esi               ; n a float
12    divss   xmm0, xmm1               ; mean = suma(arr) / n
13    movss   [rbx], xmm0              ; guardar mean

```

Listing 3: Fragmento de `compute_stats` (versión escalar): respaldo de registros antes de `call sum_array`

La varianza poblacional se calcula con un segundo recorrido explícito del arreglo, también acumulando en `double`: la media (un `float`) se convierte a `double` una sola vez antes del bucle, y en cada iteración se convierte `arr[i]` a `double`, se calcula $(x - \mu)^2$ con `subsd/mulsd` y se acumula con `addsd`. Al final, la división entre `n` se hace en `double` y el resultado se convierte a `float` al guardarlo:

```

1     xor     eax, eax
2     xorpd   xmm0, xmm0              ; acumulador EN DOUBLE = 0.0
3     cvtss2sd xmm2, [rbx]            ; mean (float) -> double, se lee una sola
4                                     ; vez y se reutiliza en cada iteracion
5  .var_loop:
6     cmp     eax, esi
7     jge     .var_done
8     cvtss2sd xmm1, [rdi + rax*4]    ; arr[i]: float -> double
9     subsd   xmm1, xmm2              ; (arr[i] - mean), en double
10    mulsd   xmm1, xmm1              ; (arr[i] - mean)^2, en double
11    addsd   xmm0, xmm1              ; acumulador += eso, en double
12    inc     eax
13    jmp     .var_loop
14  .var_done:
15    cvtsi2sd xmm1, esi               ; n a double
16    divsd   xmm0, xmm1              ; var = acumulador / n, en double
17    cvtsd2ss xmm0, xmm0              ; convierto el resultado final a float
18    movss   [rcx], xmm0              ; guardar var

```

Listing 4: Fragmento de `compute_stats` (versión escalar): varianza acumulada en double

El mínimo y el máximo se obtienen en un tercer recorrido, inicializando ambos con `arr[0]` y actualizándolos con comiss más saltos condicionales (`jb` / `ja`). Esta parte no requiere precisión extendida, ya que no involucra sumas acumulativas:

```

1  movss    xmm0, [rdi]          ; min = arr[0]
2  movss    xmm1, [rdi]          ; max = arr[0]
3  mov     eax, 1                ; continuar desde arr[1]
4
5  .min_max_loop:
6  cmp     eax, esi              ; comparo i con n
7  jge     .min_max_done         ; termina si i >= n
8  movss    xmm2, [rdi + rax*4]  ; arr[i]
9  comiss   xmm2, xmm0           ; comparo arr[i] con min
10 jb     .update_min           ; si arr[i] < min, actualiza min
11 comiss   xmm2, xmm1           ; comparo arr[i] con max
12 ja     .update_max           ; si arr[i] > max, actualiza max
13 inc     eax
14 jmp     .min_max_loop
15 .update_min:
16 movss    xmm0, xmm2           ; min = arr[i]
17 inc     eax
18 jmp     .min_max_loop
19 .update_max:
20 movss    xmm1, xmm2           ; max = arr[i]
21 inc     eax
22 jmp     .min_max_loop

```

Listing 5: Fragmento de `compute_stats` (versión escalar): cálculo de mínimo y máximo

El caso borde $n = 0$ se resuelve al inicio de la función con un salto a `.zero_case`, que escribe 0.0 en `mean`, `var`, `min` y `max` sin ejecutar ningún bucle, evitando así la división por cero. En la práctica, con el driver actual este caso nunca llega a ejecutarse: `read_input()` ya rechaza $N = 0$ como error controlado antes de reservar memoria o llamar a cualquier kernel (Sección 6); la rama `.zero_case` queda como una segunda capa de protección, para que `compute_stats` nunca divida entre cero si en algún momento se llamara directamente con $n = 0$ desde otro contexto.

4.3. normalize_array

`mean` y `stddev` llegan en `xmm0` y `xmm1` y se copian a `xmm8` y `xmm9` (registros que no se usan para pasar argumentos en System V) para que sobrevivan intactos durante todo el bucle. Antes de normalizar se compara `stddev` contra 0.0 con `comiss`: si `stddev = 0`, la función salta a un bucle alterno (`.copy_in`) que simplemente copia `in[i]` en `out[i]`, evitando la división por cero exigida como caso borde en la sección 2.3 del enunciado. En caso contrario, cada elemento se normaliza con `subss` seguido de `divss`. Esta función se mantiene en `float` sin cambios, ya que no involucra ninguna suma acumulativa sensible a precisión.

```

1  movaps    xmm8, xmm0          ; guardar mean
2  movaps    xmm9, xmm1          ; guardar stddev
3  xor     eax, eax              ; i = 0
4  xorps     xmm2, xmm2          ; 0.0 para comparar
5  comiss    xmm9, xmm2          ; stddev == 0.0 ?
6  je       .copy_in             ; si stddev = 0 copiar sin normalizar
7  .normalize_loop:
8  cmp     eax, edx
9  jge     .normalize_done
10 movss    xmm2, [rdi + rax*4]  ; in[i]
11 subss    xmm2, xmm8           ; in[i] - mean
12 divss    xmm2, xmm9           ; (in[i] - mean) / stddev
13 movss    [rsi + rax*4], xmm2  ; out[i] = ...
14 inc     eax
15 jmp     .normalize_loop
16 .copy_in:

```

```

17     cmp     eax, edx
18     jge     .normalize_done
19     movss   xmm2, [rdi + rax*4]
20     movss   [rsi + rax*4], xmm2
21     inc     eax
22     jmp     .copy_in
23
24 .normalize_done:
25     ret

```

Listing 6: `normalize_array` (versión escalar): caso borde `stddev = 0` y bucle de normalización

5. Implementación vectorial (AVX2)

Esta sección describe la implementación vectorial con AVX2 de los tres kernels: la elección de la extensión, la acumulación en `double`, la reducción horizontal y el manejo del remanente.

5.1. Elección de AVX2 frente a SSE y AVX-512

Se seleccionó AVX2 por su compatibilidad con el procesador del entorno de pruebas y por permitir trabajar con registros YMM de 256 bits. Estos registros contienen ocho valores `float32` o cuatro valores `float64`, mientras que los registros XMM de 128 bits utilizados por SSE contienen cuatro valores `float32` o dos valores `float64`.

La implementación vectorial procesa ocho entradas por iteración. Para la suma y la varianza, las convierte en dos grupos de cuatro valores de doble precisión, sobre los que realiza las operaciones de acumulación. El mínimo, el máximo y la normalización trabajan directamente con ocho valores de precisión simple.

El número de elementos procesados por instrucción no establece un límite universal de aceleración de $4\times$ para SSE ni de $8\times$ para AVX2. El rendimiento también depende de las instrucciones ejecutadas, las dependencias entre operaciones, el control de los bucles, las conversiones y el acceso a memoria. Por ello, la aceleración se determina mediante los tiempos medidos en la Sección 7.

AVX-512 permitiría utilizar registros de 512 bits y máscaras para procesar los elementos restantes. Sin embargo, el Intel Core i7-8550U del entorno de pruebas no dispone de esta extensión, por lo que no se seleccionó para el proyecto.

La codificación VEX permite expresar muchas operaciones con tres operandos, conservando los valores fuente y escribiendo el resultado en un registro independiente. En esta implementación, los elementos que quedan fuera de los bloques completos se procesan mediante una cola escalar. Las cargas y escrituras con `vmovaps` sobre registros YMM requieren alineación de 32 bytes, garantizada por las reservas de memoria del *driver*.

5.2. `sum_array`: patrón base reutilizado

El patrón que se repite en los demás kernels es: (1) inicializar los acumuladores vectoriales en cero (dos registros YMM de 4 `doubles`: `ymm0` para los elementos 0–3 de cada bloque y `ymm2` para los elementos 4–7), (2) calcular el límite vectorial con `and ecx, ~7`, (3) recorrer el arreglo de 8 en 8: dos `vcvtps2pd` convierten 4 `floats` cada una a 4 `doubles` [2, Vol. 2] (cargas de 16 bytes, que no exigen alineación) y dos `vaddpd` los acumulan por carril, (4) reducir los acumuladores a un único escalar mediante una reducción horizontal en `double` (Sección 5.4), (5) procesar el remanente ($n \bmod 8$) con un bucle escalar en `double` (`vcvtss2sd + vaddsd`) y (6) convertir el resultado a `float` con `vcvtss2ss`. Al final de la función se llama `vzeroupper` para evitar la penalización de transición AVX/SSE al retornar a código que use registros XMM.

```

1  .sum_vec_loop:
2      cmp     eax, ecx
3      jge     .sum_reduce
4      vcvtps2pd ymm1, oword [rdi + rax*4]          ; floats 0-3 -> 4 doubles
5      vcvtps2pd ymm3, oword [rdi + rax*4 + 16]      ; floats 4-7 -> 4 doubles
6      vaddpd    ymm0, ymm0, ymm1                    ; acumula por carril (elementos 0-3)
7      vaddpd    ymm2, ymm2, ymm3                    ; acumula por carril (elementos 4-7)
8      add      eax, 8
9      jmp     .sum_vec_loop
10 .sum_reduce:
11     vaddpd    ymm0, ymm0, ymm2                    ; [e0+e4, e1+e5, e2+e6, e3+e7]
12     vextractf128 xmm2, ymm0, 1                    ; mitad alta: [e2+e6, e3+e7]
13     vaddpd    xmm0, xmm0, xmm2                    ; [(e0+e4)+(e2+e6), (e1+e5)+(e3+e7)]
14     vhaddpd    xmm0, xmm0, xmm0                    ; xmm0[0] = suma total, en double
15 .sum_scalar_tail:
16     cmp      eax, esi
17     jge      .sum_done
18     vcvts2sd  xmm1, xmm1, [rdi + rax*4]          ; remanente: float -> double
19     vaddsd    xmm0, xmm0, xmm1
20     inc      eax
21     jmp     .sum_scalar_tail
22 .sum_done:
23     vcvtsd2ss xmm0, xmm0, xmm0                    ; double -> float
24     vzeroupper
25     ret

```

Listing 7: sum_array (AVX2): acumulación en double y reducción horizontal

5.3. compute_stats: media, varianza y mínimo/máximo

La función se organiza en tres fases secuenciales sobre el mismo arreglo. La media se calcula reutilizando `sum_array` exactamente con la misma estrategia de respaldo en registros callee-saved que en la versión escalar (Sección 4), ya que `call` destruye los registros callee-saved sin importar si el resto del código es escalar o vectorial. Como `sum_array` devuelve un `float`, la división entre n se hace en `float` con `vdivss`.

Para la varianza, la media (un `float`) se convierte a `double` con `vcvtss2sd` y se replica en los 4 carriles con `vbroadcastsd`. En cada iteración se convierten los 8 elementos a `double`, se calcula $(x - \mu)^2$ con `vsubpd` y `vmulpd`, y se acumula en dos registros (`ymm0` y `ymm4`). Al terminar el bucle, los acumuladores se reducen con la misma secuencia horizontal en `double` usada en `sum_array`. La división entre n se hace en `double` y el resultado se convierte a `float` al guardarlo:

```

1  xor      eax, eax
2  vxorpd    ymm0, ymm0, ymm0                      ; acumulador double (elementos 0-3)
3  vxorpd    ymm4, ymm4, ymm4                      ; acumulador double (elementos 4-7)
4  vcvts2sd  xmm1, xmm1, [rdi]                    ; mean (float) -> double
5  vbroadcastsd ymm1, xmm1                        ; mean en los 4 carriles double
6  .var_loop:
7      cmp     eax, r10d
8      jge     .var_reduce
9      vcvtps2pd ymm2, oword [rdi + rax*4]          ; floats 0-3 -> doubles
10     vcvtps2pd ymm3, oword [rdi + rax*4 + 16]      ; floats 4-7 -> doubles
11     vsubpd    ymm2, ymm2, ymm1                    ; (x - mean)
12     vsubpd    ymm3, ymm3, ymm1
13     vmulpd    ymm2, ymm2, ymm2                    ; (x - mean)^2
14     vmulpd    ymm3, ymm3, ymm3
15     vaddpd    ymm0, ymm0, ymm2                    ; acumular la suma de los cuadrados
16     vaddpd    ymm4, ymm4, ymm3
17     add      eax, 8
18     jmp     .var_loop
19 .var_reduce:

```

```

20    vaddpd   ymm0, ymm0, ymm4           ; une los dos acumuladores
21    vextractf128 xmm2, ymm0, 1        ; mitad alta (carriles 2-3)
22    vaddpd   xmm0, xmm0, xmm2         ; 2 sumas parciales
23    vhaddpd  xmm0, xmm0, xmm0         ; xmm0[0] = suma de cuadrados, en double

```

Listing 8: `compute_stats` (AVX2): varianza acumulada en `double`

El límite vectorial de esta fase se guarda en `r10d` y no en `ecx`: en este punto `rcx` ya contiene el puntero `var*` recuperado antes de la llamada a `sum_array`, y escribir sobre `ecx` pondría a cero los 32 bits altos de `rcx` (toda escritura de 32 bits en x86-64 hace zero-extend del registro de 64 bits completo), destruyendo el puntero.

Para el mínimo y el máximo, en vez de inicializar con $+\infty/-\infty$ (lo que exigiría una constante en una sección `.rodata` aparte), ambos se inicializan replicando `arr[0]` en los 8 carriles con `vbroadcastss`; esto es válido porque el caso $n = 0$ ya se descartó al inicio de `compute_stats`, así que `arr[0]` siempre es una lectura segura. El bucle principal usa `vminps/vmaxps` por carril, seguido de una reducción horizontal (ver Sección 5.4) y un remanente escalar con `vminss/vmaxss`.

5.4. Estrategia de reducción y precisión numérica

Acumular en `float32` resultó insuficiente en dos situaciones. Primero, en la varianza con N grande: cada carril acumula cuadrados del orden de 10^{10} y descarta los sumandos que quedan por debajo de su precisión, de modo que para $N = 5 \times 10^7$ la varianza vectorial daba 3314.34 frente a 3332.71 esperado (error relativo de 5.5×10^{-3} , muy por encima de la tolerancia de 10^{-4}). Segundo, en sumas con cancelación: con valores como 10^8 , 1 y -10^8 el sumando pequeño se pierde en `float` y la suma da 0 en lugar de 1. Ambos problemas desaparecen acumulando en `double`, cuya mantisa de 53 bits conserva esos sumandos: el error relativo queda limitado, en la práctica, por la conversión final a `float` (del orden de 10^{-8} – 10^{-7}).

Antes de adoptar esta solución se probaron sumas compensadas (TwoSum en la suma y Kahan en la varianza). Funcionaban, pero añadían varias instrucciones aritméticas empaquetadas por iteración y complicaban la reducción (había que reducir también los residuos). La conversión a `double` es más simple y mantiene la reducción en un solo conjunto de acumuladores.

La reducción de los acumuladores es **horizontal** y se realiza en `double`. Primero se suman los dos registros (`vaddpd ymm0, ymm0, ymm2`), lo que deja en cada carril la suma de dos posiciones del bloque, $[e_0 + e_4, e_1 + e_5, e_2 + e_6, e_3 + e_7]$. Luego `vextractf128` baja la mitad alta, `vaddpd` la suma con la mitad baja, y un único `vhaddpd` combina las dos sumas parciales restantes. El orden de suma resulta así en árbol, $((e_0 + e_4) + (e_2 + e_6)) + ((e_1 + e_5) + (e_3 + e_7))$, y no coincide con el recorrido secuencial del bucle escalar. Esa diferencia queda muy por debajo de la tolerancia de 10^{-4} : ambos recorridos comparten los acumuladores por carril y solo difieren en las últimas siete sumas, que se hacen en `double`. Tras la conversión final a `float` los resultados coinciden salvo, en casos raros, por una unidad en la última cifra de `float`, o cuando la suma casi se cancela por completo.

Durante el desarrollo también se probó una reducción **secuencial**: volcar el acumulador a la pila con `vmovupd` y sumar sus elementos uno por uno con `vaddsd`. Con acumuladores en `double`, esa variante produjo los mismos resultados que la reducción horizontal en la batería de pruebas y en las mediciones de rendimiento. La mejora de precisión se debe, por tanto, a acumular en `double` y no al orden de la reducción. Se adoptó la reducción horizontal por ser más corta, por no requerir reservar espacio en la pila ni acceder a memoria, y por reutilizar el mismo patrón (`vextractf128` más una operación horizontal) que ya se emplea para el mínimo y el máximo. Una limitación de la técnica es que `double` no es exacta: sumas con cancelaciones que superen sus 53 bits de mantisa (por ejemplo $10^{30} + 1 - 10^{30}$) seguirían perdiendo sumandos.

Para el mínimo y el máximo también se usa **reducción horizontal**, y en este caso el orden es irrelevante porque `min` y `max` son operaciones asociativas y conmutativas: el resultado final no depende del orden en que se comparen los elementos. Se usa `vextractf128` para bajar la mitad alta (carriles 4-7)

y compararla con la mitad baja, y luego `vshufps` para permutar e ir comparando los carriles restantes de a pares, ya que AVX2 no ofrece una instrucción horizontal de mínimo/máximo equivalente a `vhaddps`:

```

1  .min_max_reduce:
2      vextractf128 xmm2, ymm0, 1      ; mitad alta (carriles 4-7)
3      vminps  xmm0, xmm0, xmm2      ; min(min[0-3], min[4-7])
4      vextractf128 xmm3, ymm1, 1
5      vmaxps  xmm1, xmm1, xmm3      ; max(max[0-3], max[4-7])
6      vshufps xmm2, xmm0, xmm0, 0xB1 ; intercambia carriles 0<->1, 2<->3
7      vminps  xmm0, xmm0, xmm2
8      vshufps xmm2, xmm0, xmm0, 0x4E ; intercambia carriles 0<->2, 1<->3
9      vminps  xmm0, xmm0, xmm2      ; xmm0[0] = min de los 8 originales
10     ; analogo para xmm1/max con vmaxps

```

Listing 9: `compute_stats` (AVX2): reducción horizontal de mínimo y máximo

5.5. Versiones originales y evolución del código

El desarrollo se realizó en dos etapas. En la primera se implementaron ambos kernels, escalar y vectorial, con acumulación en `float`. En la versión vectorial, la suma y la varianza usaban un acumulador de ocho carriles `float` y una reducción horizontal con `vextractf128` y `vhaddps`. Estas versiones funcionaban correctamente con tamaños pequeños y medianos.

En la segunda etapa, las pruebas con N grande y con sumas con cancelación mostraron que esa acumulación en `float` no cumplía la tolerancia respecto de la referencia (Sección 5.4); el caso decisivo fue $N = 5 \times 10^7$. Para corregirlo se acumuló en `double` en la suma y en la varianza de ambas versiones. El mínimo, el máximo y la normalización se mantuvieron en `float`, porque no involucran sumas acumulativas. Entre esas dos etapas se probaron además una reducción secuencial y sumas compensadas, que se descartaron por las razones ya expuestas. La versión final usa acumuladores en `double` con reducción horizontal.

Las versiones originales se conservan sin modificar en la carpeta **Códigos originales (Sin doble precisión)** del repositorio, acompañadas de una nota que explica el cambio. Se incluyen únicamente como evidencia del desarrollo: todos los resultados de este informe, incluidas las pruebas de la Sección 6 y las mediciones de la Sección 7, corresponden a las versiones finales descritas en las Secciones 4 y 5.

5.6. Manejo del remanente (*tail handling*)

En cada kernel, el límite del bucle vectorial se obtiene con `and reg, ~7`, que redondea n hacia abajo al múltiplo de 8 inmediatamente inferior (equivalente a $\lfloor n/8 \rfloor \times 8$). Los $n \bmod 8$ elementos que sobran se procesan en un bucle de cierre aparte, sin requerir alineación de memoria. En `sum_array` y en la varianza el remanente continúa la acumulación en `double` con las versiones escalares de las instrucciones (`vcvtss2sd`, `vaddsd`, `vsubsd`, `vmulsd`), de modo que ningún elemento se acumula con menor precisión que el resto; en el mínimo, el máximo y la normalización se usan `vminss`, `vmaxss`, `vsubss` y `vdivss` sobre `float`. Si $n < 8$, el bucle vectorial no se ejecuta y todo el trabajo pasa al remanente; con $n = 15$, por ejemplo, el bucle vectorial procesa 8 elementos y la cola escalar los 7 restantes (Sección 8).

5.7. `normalize_array`

`mean` y `stddev` llegan como escalares en `xmm0/xmm1`; se respaldan en `xmm8/xmm9` (para usarlos en el remanente escalar) y además se replican en los 8 carriles de `ymm0/ymm1` con `vbroadcastss`. El caso borde `stddev = 0` se resuelve igual que en la versión escalar: se compara `stddev` contra un registro puesto en cero con `vxorps + vcomiss`, y si son iguales se salta a un bucle que copia `in[i]` en `out[i]` sin dividir. En caso contrario, el bucle vectorial resta y divide 8 elementos a la vez (`vsubps/vdivps`) con carga y almacenamiento alineados (`vmovaps`), y el remanente se procesa con las versiones escalares equivalentes:

```

1  vbroadcastss ymm0, xmm0 ; mean en los 8 carriles
2  vbroadcastss ymm1, xmm1 ; stddev en los 8 carriles
3  vxorps xmm2, xmm2, xmm2
4  vcomiss xmm9, xmm2 ; stddev == 0.0 ?
5  je .stddev_zero_case
6  .normalize_loop:
7  cmp eax, r10d
8  jge .normalize_scalar_tail
9  vmovaps ymm2, [rdi+rax*4] ; carga 8 floats de in
10 vsubps ymm2, ymm2, ymm0 ; (in[i] - mean)
11 vdivps ymm2, ymm2, ymm1 ; (in[i] - mean) / stddev
12 vmovaps [rsi+rax*4], ymm2 ; guarda en out
13 add eax, 8
14 jmp .normalize_loop

```

Listing 10: `normalize_array` (AVX2): broadcast de mean/stddev y bucle vectorial

6. Casos de prueba

La verificación funcional comprende entradas aleatorias con tamaños $N = 0, 1, 7, 8, 15, 16$ y 1000, además de casos con valores constantes, valores negativos, suma igual a cero y magnitudes extremas. Esta selección permite comprobar arreglos pequeños, bloques vectoriales completos y tamaños con remanente. Las entradas aleatorias se generan con `tools/gen_input.py`, utilizando la semilla 123 para reproducir los resultados. La suite de `tools/verify_reference.py` incorpora también los casos negativos y de suma cero.

La referencia de aceptación se calcula mediante NumPy en precisión simple (`float32`). Se comprueban la suma, la media, la varianza poblacional, la desviación estándar, el mínimo y el máximo. Además, se verifican todos los elementos del arreglo normalizado contra la referencia y se comparan las salidas de ambas implementaciones. La tolerancia relativa es de 10^{-4} ; cuando el valor de referencia es exactamente cero, se utiliza una tolerancia absoluta de 10^{-6} . El cálculo con `math.fsum` constituye una herramienta de diagnóstico de precisión y no sustituye la referencia de aceptación.

Como comprobación adicional, el verificador reconstruye la normalización en `float32` utilizando la media y la desviación estándar publicadas por cada ejecutable. Esta comprobación exige igualdad exacta de los valores del arreglo con el resultado de dicha fórmula. Cuando la desviación estándar es cero, la salida debe ser una copia de la entrada.

Para $N = 0$, ambos ejecutables rechazan la entrada de manera controlada antes de invocar los kernels. En este caso se verifica que el proceso termine con un código de error, emita el mensaje correspondiente y no genere un archivo de salida ni un resumen estadístico.

La Tabla 2 presenta los estadísticos publicados en los archivos `.stats.txt`. Los commits `befc525` y `4607d74` incorporan las entradas, las salidas y los resúmenes empleados para documentar estos casos. La expresión “Igual a escalar” indica coincidencia de los estadísticos mostrados; por sí sola, no demuestra igualdad exacta entre los arreglos normalizados.

El registro `benchmark/results/run-7uezs.bi/verification.log`, incorporado en el commit `7a68162`, documenta una ejecución de la suite con cero casos fallidos. Se comprobaron los estadísticos y todos los elementos de los arreglos normalizados contra la referencia NumPy `float32` y entre ambas implementaciones. También se verificó la normalización utilizando la media y la desviación estándar publicadas por cada ejecutable. La corrida registra el commit `442f726` como versión evaluada. Esta evidencia corresponde a las entradas y salidas generadas durante esa ejecución; no constituye una comprobación individual de los archivos archivados en `data/tabla2`.

Además, los registros `verification_1000.log`, `verification_100000.log`, `verification_1000000.log` y `verification_500000000.log`, ubicados en la misma carpeta, documentan la verificación satisfactoria de cada tamaño utilizado en las mediciones de rendimiento, incluidos los arreglos normalizados completos.

Tabla 2: Casos de prueba y resultados obtenidos

Caso	N	Entrada	Salida escalar: suma; media; varianza; desviación; mínimo; máximo	Salida vectorial	Resultado ¹
Arreglo vacío	0	Sin elementos	Rechazo controlado, código 1; sin archivo de salida ni resumen.	Rechazo controlado	PASA
Un elemento	1	Aleatoria	$-89,5272827; -89,5272827; 0; 0; -89,5272827; -89,5272827$	Igual a escalar ²	PASA
Menor que un bloque	7	Aleatoria	$-273,990631; -39,1415176; 3661,48193; 60,5101814; -92,3692703; 80,2397766$	Igual a escalar ²	PASA
Un bloque completo	8	Aleatoria	$-307,551086; -38,4438858; 3207,20361; 56,6321793; -92,3692703; 80,2397766$	Igual a escalar ²	PASA
Bloque con remanente	15	Aleatoria	$-534,37677; -35,6251183; 2859,59399; 53,4751701; -99,6658859; 80,2397766$	Igual a escalar ²	PASA
Dos bloques completos	16	Aleatoria	$-616,854065; -38,5533791; 2809,49023; 53,0046234; -99,6658859; 80,2397766$	Igual a escalar ²	PASA
Valores iguales	16	Todos los valores = 5	80; 5; 0; 0; 5; 5. La salida normalizada es una copia de la entrada.	Igual a escalar ²	PASA
Valores negativos	17	$-1, -2, \dots, -17$	$-153; -9; 24; 4,89897966; -17; -1$	Igual a escalar ²	PASA
Suma que cancela a cero	9	$-4, -3, \dots, 4$	$0; 0; 6,66666651; 2,58198881; -4; 4$	Igual a escalar ²	PASA
Valores extremos con remanente	15	Patrón edge ³	$-1000000; -66666,6641; 3,28888877 \times 10^{11}; 573488,312; -1000000; 1000000$	Igual a escalar ²	PASA
Valores extremos sin remanente	16	Patrón edge ³	$0; 0; 3,75000007 \times 10^{11}; 612372,438; -1000000; 1000000$	Igual a escalar ²	PASA
Tamaño intermedio	1000	Aleatoria	$-2830,93115; -2,83093119; 3305,87793; 57,4967651; -99,9303207; 99,982666$	Igual a escalar ²	PASA

¹ El estado **PASA** corresponde a los casos ejecutados por la suite cuyo registro se encuentra en `benchmark/results/run-7uezsbi/verification.log`, *concerocasos fallidos. Esta ejecución verificó*. Se utilizó una tolerancia relativa de 10^{-4} y una tolerancia absoluta de 10^{-6} únicamente cuando el valor de referencia era cero.

² ‘‘Igual a escalar’’ indica que los seis estadísticos publicados, suma, media, varianza, desviación estándar, mínimo y máximo, coinciden en los resúmenes `.stats.txt` de ambas versiones.

³ El patrón edge repite la secuencia $(-10^6, 10^6, 0, -10^{-4}, 10^{-4}, -1, 1)$ hasta completar N elementos.

7. Resultados de rendimiento

Los tiempos corresponden a la corrida `benchmark/results/run-7uezs.bi/`, mientras que los contadores de hardware proceden de `benchmark/results/perf/run-26o_9uod/`. Ambas registran el commit 442f726 como versión evaluada y utilizan los mismos binarios. Los resultados fueron incorporados al repositorio mediante el commit 7a68162.

7.1. Tiempos de ejecución

Se midió el tiempo conjunto de `sum_array`, `compute_stats` y `normalize_array` mediante `clock_gettime`. Cada proceso realizó una ejecución de calentamiento no cronometrada antes de obtener una muestra. Para cada tamaño e implementación se realizaron 30 mediciones, precedidas por una verificación funcional.

La Tabla 3 presenta el promedio y la desviación estándar muestral de los tiempos, expresados en milisegundos. El intervalo cronometrado comprende los kernels y excluye la lectura del archivo, la reserva de memoria y la escritura de los resultados.

La implementación vectorial obtuvo menores tiempos promedio en los cuatro tamaños. Para $N = 10^3$, la desviación estándar representa aproximadamente el 16,4 % del promedio escalar y el 9,3 % del vectorial. Debido a la corta duración de estas ejecuciones, las variaciones del entorno y el costo de medición pueden influir considerablemente en el resultado. Para $N = 5 \times 10^7$, la versión vectorial también presenta una dispersión apreciable, cercana al 19,3 % de su promedio.

Tabla 3: Tiempo promedio $\pm$ desviación estándar por tamaño y versión (ms, 30 repeticiones)

N	Escalar (ms)	Vectorial (ms)	Speedup
10^3	$0,015930 \pm 0,002610$	$0,007343 \pm 0,000681$	$2,17\times$
10^5	$0,741517 \pm 0,042494$	$0,079527 \pm 0,007304$	$9,32\times$
10^6	$7,594383 \pm 0,251190$	$1,137517 \pm 0,126845$	$6,68\times$
5×10^7	$395,712819 \pm 19,196657$	$80,076817 \pm 15,422137$	$4,94\times$

7.2. Speedup vs. tamaño de N

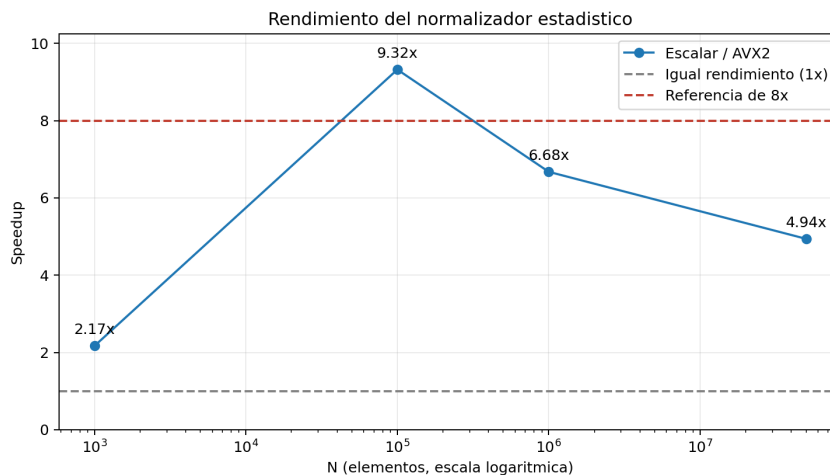


Figura 1: Speedup (tiempo escalar / tiempo vectorial) en función de N, escala logarítmica

El *speedup* se calcula dividiendo el tiempo promedio escalar entre el tiempo promedio vectorial para un mismo tamaño:

$$\text{Speedup} = \frac{T_{\text{escalar}}}{T_{\text{vectorial}}}.$$

Los resultados son **2,17×** para $N = 10^3$, **9,32×** para $N = 10^5$, **6,68×** para $N = 10^6$ y **4,94×** para $N = 5 \times 10^7$. La Figura 1 presenta estos valores.

La aceleración no aumenta de forma monótona: alcanza su máximo para $N = 10^5$ y disminuye en los tamaños mayores. Esta tendencia es compatible con una influencia creciente del acceso a memoria, aunque las mediciones no permiten atribuirla exclusivamente a una causa.

Los arreglos de entrada y salida ocupan conjuntamente aproximadamente $8N$ bytes: 8 kB, 800 kB, 8 MB y 400 MB, respectivamente. Para $N = 10^6$, esta ocupación se aproxima a los 8 MiB nominales de la caché L3 del procesador de pruebas; para $N = 5 \times 10^7$, supera ampliamente esa capacidad. La ocupación nominal por sí sola no determina qué datos permanecen en caché, pues también intervienen los recorridos de los kernels y la actividad del sistema.

El *speedup* de 9,32× no contradice el procesamiento de ocho entradas por iteración. El ancho vectorial no establece un límite universal de aceleración: también influyen la cantidad de instrucciones, las dependencias entre operaciones y el control de los bucles. Además, la suma y la varianza trabajan con dos grupos de cuatro valores `double` por bloque, mientras que el mínimo, el máximo y la normalización operan sobre ocho valores `float`.

7.3. Resultados de perf stat

Se utilizó `perf stat` con los eventos `cycles`, `instructions`, `cache-misses` y `cache-references`. La Tabla 4 presenta los resultados de tres corridas por tamaño e implementación. Para reducir el peso relativo del arranque y de la entrada/salida, el conjunto de kernels se repitió dentro de cada proceso 200000, 2000, 200 y 30 veces para $N = 10^3$, 10^5 , 10^6 y 5×10^7 , respectivamente.

Los ciclos y las instrucciones por elemento se calculan dividiendo los contadores entre N multiplicado por el número de repeticiones. El IPC se obtiene dividiendo las instrucciones totales entre los ciclos totales de las tres corridas. El porcentaje de fallos de caché se calcula como:

$$100 \times \frac{\sum \text{cache-misses}}{\sum \text{cache-references}}.$$

Tabla 4: Contadores de `perf stat` por elemento procesado, promedio de tres corridas

N	Versión	Ciclos / elemento	Instrucciones / elemento	IPC	Fallos de caché
10^3	Escalar	31,74	37,81	1,191	2,59 %
10^3	Vectorial	6,86	6,26	0,913	3,60 %
10^5	Escalar	27,01	37,02	1,370	0,19 %
10^5	Vectorial	2,79	5,39	1,935	0,05 %
10^6	Escalar	28,50	37,09	1,301	16,66 %
10^6	Vectorial	3,61	5,46	1,512	9,99 %
5×10^7	Escalar	30,12	37,57	1,247	70,68 %
5×10^7	Vectorial	5,82	5,94	1,021	60,56 %

La versión vectorial ejecuta aproximadamente entre 6 y 7 veces menos instrucciones por elemento que la escalar. Esta reducción contribuye a explicar sus menores tiempos. En la suma y la varianza utiliza conversiones a doble precisión y operaciones empaquetadas como `vaddpd`, `vsubpd` y `vmulpd`; en la normalización emplea `vsubps` y `vdivps` sobre ocho valores de precisión simple.

El IPC vectorial supera al escalar para $N = 10^5$ y $N = 10^6$: alcanza **1,935 y 1,512**, frente a **1,370 y 1,301**, respectivamente. Es menor para $N = 10^3$ y $N = 5 \times 10^7$. Por tanto, la aceleración se relaciona con la combinación de menos instrucciones y menos ciclos por elemento, sin requerir un IPC superior en todos los tamaños.

Para $N = 5 \times 10^7$, el IPC vectorial es de **1,021**, frente a **1,247** en la versión escalar. Los porcentajes de fallos de caché son de **60,56 % y 70,68 %**, respectivamente. Estos resultados son compatibles con mayores esperas asociadas al acceso a memoria, pero no demuestran por sí solos que el ancho de banda sea el único factor limitante. Tampoco representan el porcentaje de elementos del arreglo que producen un fallo de caché.

Los contadores abarcan el proceso completo, incluida la gestión de memoria y la entrada/salida. Las repeticiones internas reducen el peso relativo de estas tareas, pero no las excluyen. Por ello, la Tabla 4 complementa los tiempos de los kernels de la Tabla 3 y no representa exactamente el mismo intervalo de medición.

8. Evidencia de la sesión de depuración con GDB

La sesión de depuración se realizó con la versión final de los kernels (acumuladores en `double` y reducción horizontal). La sesión principal usa el caso pequeño $N = 16$, múltiplo de 8, en el que el bucle vectorial procesa el arreglo completo en dos iteraciones y no se ejecuta la cola escalar. Para observar el manejo del remanente se repitió el conteo de ejecuciones con $N = 15$. Las entradas se generaron con `tools/gen_input.py` en modo `random`.

Los binarios se compilaron con símbolos de depuración (`-g` en gcc y `-g -F dwarf` en NASM), lo que permite colocar breakpoints por línea del archivo `.asm`. Los números de línea se obtuvieron con `grep -n` sobre `stats_vector.asm` y `stats_scalar.asm`. GDB se lanzó con `gdb --args ./bin/norm_vector data/input_n16.dat data/out_n16.dat 1`; la opción `--args` le pasa al programa los mismos argumentos que recibiría al ejecutarlo directamente (archivo de entrada, archivo de salida y número de repeticiones). Un breakpoint por línea detiene la ejecución justo antes de esa instrucción.

Los tiempos que imprime el programa durante la sesión no se utilizan, ya que incluyen las pausas del depurador (en una de las sesiones, por ejemplo, `sum_array` reporta más de 10^5 ms). Las mediciones de rendimiento se presentan en la Sección 7. Esta sesión se ejecutó en un equipo distinto al de las Secciones 2 y 7: procesador AMD Ryzen 5 5600H (con soporte AVX2), Ubuntu 20.04 LTS, GDB 10.1, NASM 2.13.03 y GCC 10.2.0. Esa diferencia no afecta la evidencia de esta sección, que consiste en valores de registros, contenido de memoria y conteos de ejecución, y no depende de la velocidad del equipo. Los listados son extractos de la sesión: se omiten los mensajes de arranque de GDB y los comandos repetidos.

8.1. Acumuladores de `sum_array` durante el bucle vectorial

Se colocó un breakpoint en la línea 24 de `stats_vector.asm`, la instrucción `add eax, 8` que sigue a las dos `vaddpd` del bucle (líneas 22 y 23). En cada parada, los acumuladores ya reflejan una iteración completa. Como acumulan `double`, se leen con `v4.double`. Con `x/16fw $rdi` se muestran además los 16 float de entrada, ya que `rdi` contiene el puntero al arreglo:

```

1 (gdb) break stats_vector.asm:24
2 Breakpoint 1 at 0x3bca: file asm/vector/stats_vector.asm, line 24.
3 (gdb) run
4 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:24
5 24      add      eax, 8
6 (gdb) x/i $pc
7 => 0x555555557bca <sum_array.sum_vec_loop+23>: add      eax,0x8
8 (gdb) x/16fw $rdi
9 0x55555555c200: -42.2642708      -78.5689087      74.8392944      -10.674119
10 0x55555555c210: -5.94248104      -99.9967117      -75.2702713      -77.586792
11 0x55555555c220: 75.9329071      70.856781      -22.635313      16.237875

```

```

12 0x55555555c230: -15.2736158      -97.1709213      96.0412979      -38.6728249
13 (gdb) print $ymm0.v4_double
14 $1 = {-42.264270782470703, -78.56890869140625, 74.83929443359375, -10.674118995666504}
15 (gdb) print $ymm2.v4_double
16 $2 = {-5.9424810409545898, -99.996711730957031, -75.270271301269531, -77.5867919921875}
17 (gdb) continue
18 Breakpoint 1, sum_array.sum_vec_loop () at asm/vector/stats_vector.asm:24
19 24      add     eax, 8
20 (gdb) print $ymm0.v4_double
21 $3 = {33.668636322021484, -7.712127685546875, 52.203981399536133, 5.563755989074707}
22 (gdb) print $ymm2.v4_double
23 $5 = {-21.216096878051758, -197.16763305664062, 20.771026611328125, -116.25961685180664}

```

Listing 11: Acumuladores ymm0 y ymm2 en las dos iteraciones del bucle de sum_array ($N = 16$)

En la primera parada, ymm0 contiene los elementos 0 a 3 del arreglo y ymm2 los elementos 4 a 7, convertidos a double por vcvtps2pd: son los mismos valores del arreglo de entrada, con más cifras significativas. En la segunda parada, cada carril de ymm0 acumula $x_j + x_{j+8}$ y cada carril de ymm2 acumula $x_{j+4} + x_{j+12}$. Por ejemplo, el carril 0 de ymm0 vale $-42,2643 + 75,9329 = 33,6686$, y el carril 1 de ymm2 vale $-99,9967 + (-97,1709) = -197,1676$. Cada iteración usa dos vaddpd sobre cuatro double, de modo que se procesan ocho elementos por iteración.

8.2. Reducción horizontal en double

Para observar la reducción se colocaron breakpoints en la línea 31 (antes de unir los dos acumuladores), en la línea 34 (antes de vhaddpd) y en la línea 47 (después de convertir el resultado a float):

```

1 (gdb) break stats_vector.asm:31
2 Breakpoint 2 at 0x3bcf: file asm/vector/stats_vector.asm, line 31.
3 (gdb) continue
4 Breakpoint 2, sum_array.sum_reduce () at asm/vector/stats_vector.asm:31
5 31      vaddpd  ymm0, ymm0, ymm2
6 (gdb) print $ymm0.v4_double
7 $6 = {33.668636322021484, -7.712127685546875, 52.203981399536133, 5.563755989074707}
8 (gdb) print $ymm2.v4_double
9 $7 = {-21.216096878051758, -197.16763305664062, 20.771026611328125, -116.25961685180664}
10 (gdb) tbreak stats_vector.asm:34
11 Temporary breakpoint 3 at 0x555555557bdd: file asm/vector/stats_vector.asm, line 34.
12 (gdb) continue
13 Temporary breakpoint 3, sum_array.sum_reduce () at asm/vector/stats_vector.asm:34
14 34      vhaddpd xmm0, xmm0, xmm0
15 (gdb) print $xmm0.v2_double
16 $8 = {85.427547454833984, -315.57562160491943}
17 (gdb) stepi
18 sum_array.sum_scalar_tail () at asm/vector/stats_vector.asm:38
19 38      cmp     eax, esi
20 (gdb) print $xmm0.v2_double
21 $9 = {-230.14807415008545, -230.14807415008545}
22 (gdb) tbreak stats_vector.asm:47
23 Temporary breakpoint 4 at 0x555555557bf6: file asm/vector/stats_vector.asm, line 47.
24 (gdb) continue
25 Temporary breakpoint 4, sum_array.sum_done () at asm/vector/stats_vector.asm:47
26 47      vzeroupper
27 (gdb) print $xmm0.v4_float[0]
28 $10 = -230.148071

```

Listing 12: Reducción horizontal de sum_array y conversión final a float

En la línea 31, ymm0 y ymm2 conservan los valores finales del bucle. Las tres instrucciones siguientes (vaddpd, vextractf128 y vaddpd) los reducen a dos sumas parciales: 85,4275 y $-315,5756$, que corresponden al orden en árbol $((e_0 + e_4) + (e_2 + e_6))$ y $((e_1 + e_5) + (e_3 + e_7))$. La instrucción vhaddpd las

suma y deja el total, $-230,148074\dots$, en los dos carriles de `xmm0`. Finalmente, `vcvtsd2ss` lo convierte a `float` y el resultado, $-230,148071$, coincide con la suma que imprime el programa.

8.3. Varianza acumulada en double

Se colocaron breakpoints en la línea 124 (`add eax, 8` del bucle de la varianza), en la línea 132 (`vhaddpd`) y en la línea 147 (conversión del resultado a `float`):

```

1 (gdb) break stats_vector.asm:124
2 (gdb) break stats_vector.asm:132
3 (gdb) break stats_vector.asm:147
4 (gdb) run
5 Breakpoint 1, compute_stats.var_loop () at asm/vector/stats_vector.asm:124
6 124      add    eax, 8                ; Incrementar el indice en 8
7 (gdb) print $ymm1.v4_double
8 $1 = {-14.384254455566406, -14.384254455566406, -14.384254455566406,
9      -14.384254455566406}
10 (gdb) print $ymm0.v4_double
11 $2 = {777.29531038845016, 4119.6698393743136, 7960.8416763763526, 13.76510513080666}
12 (gdb) print $ymm4.v4_double
13 $3 = {71.263538383646846, 7329.4928407305852, 3707.1070473352447, 3994.5607510679984}
14 (gdb) continue
15 Breakpoint 1, compute_stats.var_loop () at asm/vector/stats_vector.asm:124
16 (gdb) print $ymm0.v4_double
17 $4 = {8934.4849826541758, 11385.703965910361, 8028.921644042046, 951.47991658976116}
18 (gdb) print $ymm4.v4_double
19 $5 = {72.054502050605151, 14183.125052194344, 15900.909663149389, 4584.4954033406248}
20 (gdb) continue
21 Breakpoint 2, compute_stats.var_reduce () at asm/vector/stats_vector.asm:132
22 (gdb) print $xmm0.v2_double
23 $6 = {32936.370791896217, 31104.80433803509}
24 (gdb) stepi
25 compute_stats.var_scalar_tail () at asm/vector/stats_vector.asm:135
26 (gdb) print $xmm0.v2_double[0]
27 $7 = 64041.175129931304
28 (gdb) continue
29 Breakpoint 3, compute_stats.var_done () at asm/vector/stats_vector.asm:147
30 147      vcvtsd2ss xmm0, xmm0        ; double -> float
31 (gdb) print $xmm0.v2_double[0]
32 $8 = 4002.5734456207065
33 (gdb) stepi
34 148      vmovss    [rcx], xmm0
35 (gdb) print $xmm0.v4_float[0]
36 $9 = 4002.57349
37 (gdb) stepi
38 152      xor     eax, eax ; i = 0
39 (gdb) x/fw $rcx
0x7fffffffdc64: 4002.57349

```

Listing 13: Varianza: acumuladores, reducción y conversión final ($N = 16$)

Los cuatro carriles de `ymm1` contienen la media ($-14,3842545$, un `float`) ya convertida a `double`. En la primera parada, `ymm0` y `ymm4` guardan $(x - \mu)^2$ para los elementos 0 a 3 y 4 a 7; por ejemplo, el carril 0 de `ymm0` vale $(-42,2643 + 14,3843)^2 = 777,2953$. En la segunda parada ya se sumaron los elementos 8 a 15. La reducción deja la suma de cuadrados, $64041,175\dots$, y su división entre $n = 16$ da $4002,5734456\dots$ en `double` (línea 147). Solo la conversión final a `float` redondea el resultado: `vcvtsd2ss` produce $4002,57349$, valor que se guarda en la dirección apuntada por `rcx` y coincide con la varianza impresa por el programa ($4002,573486$; GDB muestra nueve cifras significativas y `printf` seis decimales). Este es el comportamiento que permite cumplir la tolerancia para N grande (Sección 5.4).

8.4. Estado de ymm0 y ymm1 en normalize_array

Al colocar un breakpoint en `normalize_array` y consultar `ymm0` antes de ejecutar `vbroadcastss`, solo el carril 0 contiene un valor, la media recibida como argumento escalar en `xmm0`; los demás carriles están en cero. Un segundo breakpoint en la línea 227, justo después de los dos `vbroadcastss`, muestra la media y la desviación estándar replicadas en los ocho carriles:

```

1 (gdb) break normalize_array
2 Breakpoint 4 at 0x555555557d6f: file asm/vector/stats_vector.asm, line 221.
3 (gdb) break stats_vector.asm:227
4 Breakpoint 5 at 0x555555557d81: file asm/vector/stats_vector.asm, line 227.
5 (gdb) run
6 Breakpoint 4, normalize_array () at asm/vector/stats_vector.asm:221
7 221      movaps  xmm8, xmm0 ; Guardar mean en xmm8
8 (gdb) print $ymm0.v8_float
9 $10 = {-14.3842545, 0, 0, 0, 0, 0, 0, 0}
10 (gdb) continue
11 Breakpoint 5, normalize_array () at asm/vector/stats_vector.asm:227
12 227      xor     eax, eax ; i = 0
13 (gdb) print $ymm0.v8_float
14 $11 = {-14.3842545, -14.3842545, -14.3842545, -14.3842545, -14.3842545,
15       -14.3842545, -14.3842545, -14.3842545}
16 (gdb) print $ymm1.v8_float
$12 = {63.2658958, 63.2658958, 63.2658958, 63.2658958, 63.2658958, 63.2658958,
      63.2658958, 63.2658958}

```

Listing 14: Estado de `ymm0` y `ymm1` al entrar a `normalize_array` y tras `vbroadcastss`

El valor $-14,3842545$ coincide con la media impresa por el programa ($-230,148071/16 = -14,384254$) y $63,2658958$ con la desviación estándar ($\sigma = \sqrt{4002,573486} \approx 63,265896$). Esto confirma que ambos argumentos llegan correctamente según la convención System V.

8.5. Memoria del arreglo de salida

Se colocó un breakpoint en la línea 267, la primera instrucción después de la etiqueta `.normalize_done`, posterior al bucle de normalización. Como `out` es una variable local del *driver* en C y no es visible desde ensamblador, se usó el registro `rsi`, que según la convención System V contiene el segundo argumento (el puntero a `out`):

```

1 (gdb) break stats_vector.asm:267
2 Breakpoint 6 at 0x555555557de3: file asm/vector/stats_vector.asm, line 267.
3 (gdb) continue
4 Breakpoint 6, normalize_array.normalize_done () at asm/vector/stats_vector.asm:267
5 267      vzeroupper
6 (gdb) x/16fw $rsi
7 0x55555555b520: -0.440680027    -1.01452219    1.41029453    0.0586435311
8 0x55555555b530: 0.133433238     -1.35321653   -0.962382913  -0.998998523
9 0x55555555b540: 1.4275806       1.34734571   -0.130418748  0.484022707
10 0x55555555b550: -0.0140575171  -1.30855119   1.74541986    -0.383912534

```

Listing 15: Contenido del arreglo de salida tras el bucle de normalización ($N = 16$)

Los valores son consistentes con $y[i] = (x[i] - \mu)/\sigma$ con $\mu = -14,3842545$ y $\sigma = 63,2658958$. Por ejemplo, para $x[0] = -42,2643$ (Listado 11) se obtiene $(-42,2643 + 14,3843)/63,2659 \approx -0,4407$, y para $x[2] = 74,8393$ se obtiene $(74,8393 + 14,3843)/63,2659 \approx 1,4103$, ambos iguales a lo observado en memoria. El comando `x/16fw` muestra los 16 elementos de salida.

8.6. Comparación de iteraciones: escalar vs. vectorial

Para comparar el trabajo de ambos bucles se contó, con el mismo archivo de entrada ($N = 16$), cuántas veces se ejecuta la instrucción principal de cada kernel. Se usó `ignore` con un valor alto para

que GDB no se detuviera y solo contabilizara los cruces de cada breakpoint:

```

1  # Version vectorial
2  (gdb) break stats_vector.asm:22
3  (gdb) break stats_vector.asm:122
4  (gdb) break stats_vector.asm:170
5  (gdb) break stats_vector.asm:242
6  (gdb) ignore 1 100000          # igual para los breakpoints 2, 3 y 4
7  (gdb) run
8  [Inferior 1 (process 25109) exited normally]
9  (gdb) info breakpoints
10 1 breakpoint keep y 0x0000555555557bc2 asm/vector/stats_vector.asm:22
11   breakpoint already hit 4 times
12 2 breakpoint keep y 0x0000555555557c94 asm/vector/stats_vector.asm:122
13   breakpoint already hit 2 times
14 3 breakpoint keep y 0x0000555555557cfe asm/vector/stats_vector.asm:170
15   breakpoint already hit 2 times
16 4 breakpoint keep y 0x0000555555557da3 asm/vector/stats_vector.asm:242
17   breakpoint already hit 2 times
18
19 # Version escalar
20 (gdb) break stats_scalar.asm:21
21 (gdb) break stats_scalar.asm:73
22 (gdb) break stats_scalar.asm:91
23 (gdb) break stats_scalar.asm:143
24 (gdb) ignore 1 100000          # igual para los breakpoints 2, 3 y 4
25 (gdb) run
26 [Inferior 1 (process 25131) exited normally]
27 (gdb) info breakpoints
28 1 breakpoint keep y 0x000055555555683f asm/scalar/stats_scalar.asm:21
29   breakpoint already hit 32 times
30 2 breakpoint keep y 0x00005555555568ac asm/scalar/stats_scalar.asm:73
31   breakpoint already hit 16 times
32 3 breakpoint keep y 0x00005555555568da asm/scalar/stats_scalar.asm:91
33   breakpoint already hit 15 times
34 4 breakpoint keep y 0x000055555555693f asm/scalar/stats_scalar.asm:143
35   breakpoint already hit 16 times

```

Listing 16: Conteo de ejecuciones por bucle, versión vectorial y escalar ($N = 16$)

Tabla 5: Ejecuciones de la instrucción principal de cada bucle ($N = 16$)

Kernel	Escalar (línea)	Ejec.	Vectorial (línea)	Ejec.	Iteraciones por llamada (esc./vec.)
sum_array	addsd (21)	32	vaddpd (22)	4	16 / 2
Varianza	addsd (73)	16	vaddpd (122)	2	16 / 2
Mínimo y máximo	comiss (91)	15	vminps (170)	2	15 / 2
normalize_array	subss (143)	16	vsubps (242)	2	16 / 2

`sum_array` se invoca dos veces: una directamente desde el *driver*, para reportar la suma, y otra desde `compute_stats`, para calcular la media. Por eso sus conteos se duplican (32 y 4), mientras que los demás bucles se ejecutan en una sola llamada. Por cada llamada, el bucle escalar itera 16 veces (un elemento por iteración) y el vectorial $16/8 = 2$ veces; en el mínimo y el máximo el escalar itera 15 veces porque comienza en `arr[1]`. Cada iteración vectorial de la suma y de la varianza ejecuta además dos `vaddpd` (líneas 22–23 y 122–123), porque trabaja con dos grupos de cuatro `double`; el breakpoint cuenta solo la primera de cada par. Esta reducción de iteraciones no se traduce en una aceleración de $8\times$ en el tiempo total, ya que el kernel vectorial añade trabajo que el escalar no tiene (conversiones a `double`, reducción de los carriles, `vbroadcastss` y cola escalar), tema que se discute en la Sección 7.

Con acumulación en `double`, las salidas de ambas versiones coinciden en los seis estadísticos para esta entrada: suma $-230,148071$, media $-14,384254$, varianza $4002,573486$, desviación estándar $63,265896$,

mínimo $-99,996712$ y máximo $96,041298$. En una sesión previa, con las versiones originales que acumulaban en `float` y otra entrada, las sumas de ambas versiones diferían en las últimas cifras (29.336105 frente a 29.336117 ; Sección 5.5). El orden de suma sigue siendo distinto (secuencial en el escalar; por carriles y luego en árbol en el vectorial), pero al acumular en `double` esa diferencia desaparece tras la conversión final a `float` en este caso.

8.7. Manejo del remanente: prueba con $N = 15$

Para observar el manejo del remanente se repitió la prueba con $N = 15$, que no es múltiplo de 8: el límite vectorial es $15 \div 8 = 1$ y 7 , por lo que el bucle vectorial procesa 8 elementos en una sola iteración y los 7 restantes pasan a la cola escalar. Se colocaron ocho breakpoints con `ignore` sobre las instrucciones del bucle vectorial y de la cola de cada kernel: `vaddpd` (línea 22) y `vaddsd` (línea 41) en `sum_array`; `vaddpd` (línea 122) y `vsubsd` (línea 138) en la varianza; `vminps` (línea 170) y `vminss` (línea 200) en el mínimo y el máximo; y `vsubps` (línea 242) y `vsubss` (línea 252) en `normalize_array`:

```

1 (gdb) break stats_vector.asm:22
2 (gdb) break stats_vector.asm:41
3 (gdb) break stats_vector.asm:122
4 (gdb) break stats_vector.asm:138
5 (gdb) break stats_vector.asm:170
6 (gdb) break stats_vector.asm:200
7 (gdb) break stats_vector.asm:242
8 (gdb) break stats_vector.asm:252
9 (gdb) ignore 1 100000          # igual para los breakpoints 2 a 8
10 (gdb) run
11 N = 15
12 Suma = 6.889746
13 Media = 0.459316
14 [Inferior 1 (process 25157) exited normally]
15 (gdb) info breakpoints
16 1 breakpoint keep y 0x0000555555557bc2 asm/vector/stats_vector.asm:22
17 breakpoint already hit 2 times
18 2 breakpoint keep y 0x0000555555557bea asm/vector/stats_vector.asm:41
19 breakpoint already hit 14 times
20 3 breakpoint keep y 0x0000555555557c94 asm/vector/stats_vector.asm:122
21 breakpoint already hit 1 time
22 4 breakpoint keep y 0x0000555555557cbc asm/vector/stats_vector.asm:138
23 breakpoint already hit 7 times
24 5 breakpoint keep y 0x0000555555557cfe asm/vector/stats_vector.asm:170
25 breakpoint already hit 1 time
26 6 breakpoint keep y 0x0000555555557d4c asm/vector/stats_vector.asm:200
27 breakpoint already hit 7 times
28 7 breakpoint keep y 0x0000555555557da3 asm/vector/stats_vector.asm:242
29 breakpoint already hit 1 time
30 8 breakpoint keep y 0x0000555555557dbe asm/vector/stats_vector.asm:252
31 breakpoint already hit 7 times

```

Listing 17: Conteo de ejecuciones del bucle vectorial y de la cola escalar ($N = 15$)

Tabla 6: Ejecuciones del bucle vectorial y de la cola escalar ($N = 15$)

Función	Camino (línea)	Ejecuciones observadas	Ejecuciones esperadas
<code>sum_array</code>	Bucle vectorial, <code>vaddpd</code> (22)	2	1×2 llamadas
<code>sum_array</code>	Cola escalar, <code>vaddsd</code> (41)	14	7×2 llamadas
Varianza	Bucle vectorial, <code>vaddpd</code> (122)	1	1×1 llamada
Varianza	Cola escalar, <code>vsubsd</code> (138)	7	7×1 llamada
Mínimo y máximo	Bucle vectorial, <code>vminps</code> (170)	1	1×1 llamada
Mínimo y máximo	Cola escalar, <code>vminss</code> (200)	7	7×1 llamada
<code>normalize_array</code>	Bucle vectorial, <code>vsubps</code> (242)	1	1×1 llamada
<code>normalize_array</code>	Cola escalar, <code>vsubss</code> (252)	7	7×1 llamada

Los conteos coinciden con lo esperado: en cada llamada el bucle vectorial procesa los primeros 8 elementos en una sola iteración y la cola escalar procesa los 7 restantes, uno por iteración, sin requerir alineación. Como `sum_array` se invoca dos veces (*driver* y `compute_stats`), sus conteos se duplican, mientras que la varianza, el mínimo y el máximo, y `normalize_array` se ejecutan una sola vez. Con la versión escalar y la misma entrada, los mismos puntos de conteo dieron 30 ejecuciones en la suma (15×2 llamadas), 15 en la varianza, 14 en el mínimo y el máximo (comienza en `arr[1]`) y 15 en la normalización, es decir, un elemento por iteración sin cola. Esto confirma que ningún elemento queda sin procesar ni se procesa dos veces cuando N no es múltiplo de 8. Además, ambas versiones imprimieron los mismos resultados para esta entrada, por ejemplo suma 6,889746 y media 0,459316.

9. Conclusiones

- La vectorización con AVX2 produjo aceleraciones de $2,17 \times$ ($N = 10^3$), $9,32 \times$ ($N = 10^5$), $6,68 \times$ ($N = 10^6$) y $4,94 \times$ ($N = 5 \times 10^7$) (Tabla 3). El valor para $N = 10^3$ debe interpretarse con cautela: la desviación estándar es cercana al 16,4 % del promedio escalar y al 9,3 % del vectorial, y para $N = 5 \times 10^7$ la dispersión vectorial ronda el 19,3 %.
- El *speedup* de $9,32 \times$ supera el factor $8 \times$ del ancho vectorial sin contradecirlo: el ancho no es una cota de aceleración, ya que la versión escalar ejecuta unas 37 instrucciones por elemento frente a entre 5,4 y 6,3 de la vectorial (Tabla 4), por el control del bucle, las conversiones y las dependencias entre operaciones.
- El *speedup* no es monótono: alcanza su máximo en $N = 10^5$ y disminuye en los tamaños mayores. Esta tendencia es compatible con una influencia creciente del acceso a memoria, aunque las mediciones no permiten atribuirla exclusivamente a esa causa.
- Según `perf stat` (Tabla 4), la versión vectorial ejecuta aproximadamente entre 6 y 7 veces menos instrucciones por elemento que la escalar, lo que contribuye a explicar sus menores tiempos. Su IPC es mayor que el escalar en $N = 10^5$ y $N = 10^6$ (1,935 y 1,512 frente a 1,370 y 1,301) y menor en $N = 10^3$ y $N = 5 \times 10^7$. La aceleración no se debe, por tanto, a un IPC superior constante, sino a la combinación de menos instrucciones y menos ciclos por elemento.
- Para $N = 5 \times 10^7$ el IPC vectorial baja a 1,021 (frente a 1,247 en la versión escalar), con 60,56 % de fallos de caché frente a 70,68 % en la escalar. Estos valores son compatibles con esperas de acceso a memoria, pero por sí solos no demuestran que el ancho de banda sea el único factor limitante, y los porcentajes no equivalen a la fracción de elementos del arreglo que producen un fallo.
- Acumular en `float32` no alcanzó la tolerancia de 10^{-4} para $N = 5 \times 10^7$ ni en sumas con cancelación; acumular en `double` lo resolvió. La versión final usa acumuladores en `double` con reducción horizontal, y su resultado coincidió con el de la reducción secuencial probada antes en toda la

batería de pruebas. La mejora de precisión proviene, por tanto, de la acumulación en `double` y no del orden de la reducción (Sección 5.4).

- La sesión de GDB (Sección 8) confirmó, a nivel de registro y de memoria, que el bucle vectorial procesa ocho elementos por iteración (dos grupos de cuatro `double` en la suma y la varianza), que la reducción horizontal en `double` produce el mismo valor que imprime el programa y que el remanente se maneja correctamente cuando N no es múltiplo de 8 ($N = 15$). Con acumulación en `double`, las sumas de ambas versiones coincidieron en esa sesión ($-230,148071$ para $N = 16$ y $6,889746$ para $N = 15$).

9.1. Limitaciones observadas

- La acumulación ingenua en `float32` excedía la tolerancia de 10^{-4} en la varianza para N grande. La acumulación en `double` corrige los casos probados, pero no es exacta: sumas con cancelaciones que superen los 53 bits de mantisa de `double` (por ejemplo $10^{30} + 1 - 10^{30}$) seguirían perdiendo sumandos.
- Las versiones escalar y vectorial no tienen garantizada la igualdad bit a bit, porque suman en distinto orden (secuencial en la escalar; por carriles y luego en árbol en la vectorial). Tampoco se puede asegurar para toda entrada que la diferencia entre ambas, o respecto de la referencia, quede dentro de la tolerancia: cuando una suma tiene gran cancelación y su resultado es cercano a cero, el error relativo puede ser grande aunque el error absoluto sea pequeño. Para las entradas probadas (Sección 6) y en la sesión de GDB los resultados coincidieron, pero eso no constituye una garantía general.
- `perf` mide el proceso completo (E/S y asignación de memoria incluidas), no exclusivamente las funciones NASM; se mitigó repitiendo el kernel decenas o miles de veces por proceso, pero el aislamiento no es perfecto.
- Las mediciones se hicieron sobre una máquina virtual (WSL2), no sobre Linux nativo, lo que puede introducir ruido adicional en la medición de tiempos y en los contadores de `perf` frente a un despliegue *bare-metal*.

9.2. Trabajo futuro

- Explorar AVX-512 en hardware que lo soporte, para contrastar el uso de registros de máscara `k0--k7` frente al manejo manual del remanente usado aquí.
- Fusionar los recorridos de varianza y de mínimo/máximo en un solo bucle para reducir el número de pasadas sobre el arreglo, lo que podría aliviar el tráfico de memoria en los tamaños grandes.
- Evaluar suma compensada por carril (Kahan o TwoSum) sobre los acumuladores en `double`, para cubrir cancelaciones que superen la mantisa de `double`, y medir su costo frente a la versión actual.
- Repetir las mediciones de la Sección 7 en Linux nativo para descartar el ruido introducido por la virtualización de WSL2.
- Medir tamaños intermedios entre 10^5 y 10^6 para ubicar con más precisión el punto donde el arreglo deja de caber en L3 y el *speedup* empieza a caer.

Referencias

- [1] The NASM Development Team, *NASM – The Netwide Assembler: Manual*, <https://www.nasm.us/docs.html>, versión 2.16.01. Consultado: 30 de septiembre de 2026.

- [2] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual, Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, Intel Corporation, 2026.

A. Diagramas de arquitectura

Diagrama 1 - Arquitectura de software (driver C + libstats escalar / AVX2)

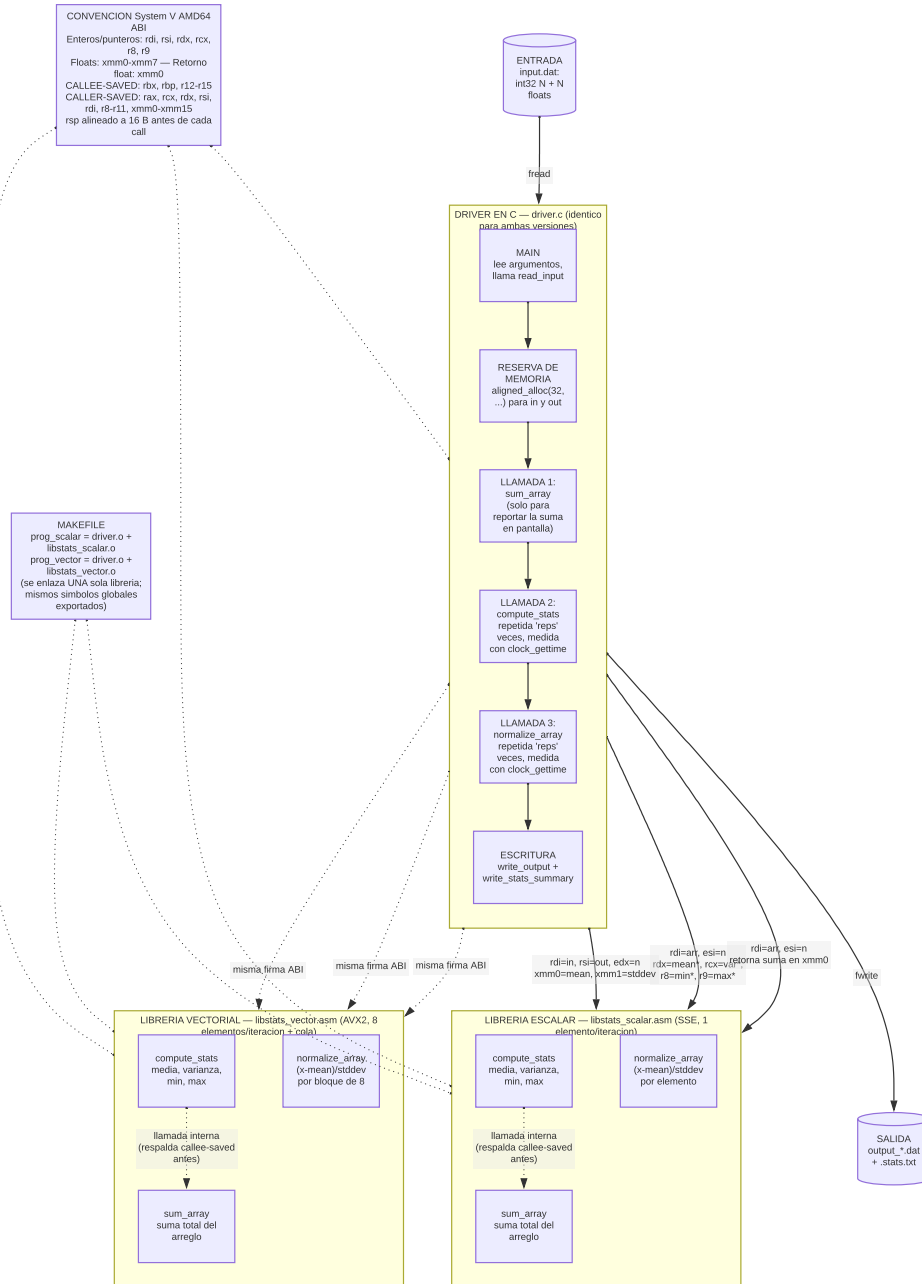


Figura 2: Diagrama 1: arquitectura de software

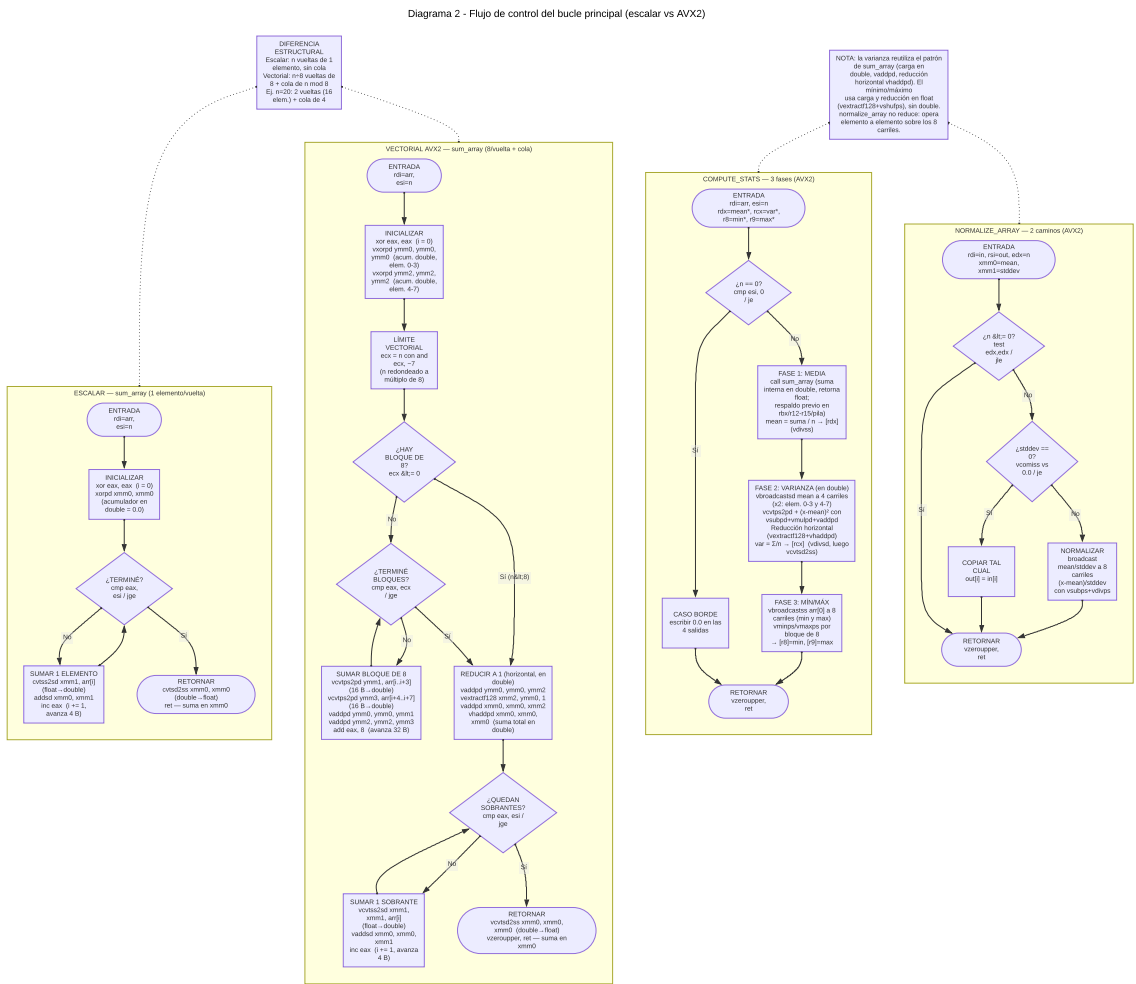


Diagrama 3 - Qué guarda cada registro en cada etapa (kernel vectorial AVX2, con la versión escalar como referencia)

ETAPA 1 - Entrada a compute_stats (si n = 0 escribe 0.0 en las 4 salidas y termina)		
Registro	En palabras simples	Contenido concreto
rdi	Recibo el arreglo de entrada	puntero arr (alineado a 32 B)
esi	Recibo el tamaño	n = cantidad de floats
rdx	Recibo dónde dejar la media	puntero mean*
rcx	Recibo dónde dejar la varianza	puntero var*
r8	Recibo dónde dejar el mínimo	puntero min*
r9	Recibo dónde dejar el máximo	puntero max*

ETAPA 2 - Respaldo antes de llamar a sum_array (el call destruye los caller-saved)		
Registro	En palabras simples	Contenido concreto
rbx	Guardo dónde va la media	copia de mean* (callee-saved)
r12	Guardo dónde va la varianza	copia de var*
r13	Guardo dónde va el mínimo	copia de min*
r14	Guardo dónde va el máximo	copia de max*
r15	Guardo el arreglo de entrada	copia de arr
pila	Guardo n (dos veces)	2 x push rsi: deja rsi alineado a 16 B

ETAPA 3 - Media: sum_array acumulando en double (8 elementos por vuelta)		
Registro	En palabras simples	Contenido concreto
rdi / esi	Recibo arreglo y tamaño	arr y n, igual que en la ABI
eax	Llevo el índice i	0, 8, 16, ... (sube de 8 en 8 = 32 B)
ecx	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0 / ymm2	Guardo dos acumuladores en double	ymm0 = elementos 0-3, ymm2 = elementos 4-7; 4 sumas parciales
ymm1 / ymm3	Guardo el bloque convertido a double	vcvtps2pd: dos cargas de 16 B, 4 floats -> 4 doubles cada una
xmm2	Uno los dos acumuladores y guardo la mitad alta	vaddpd ymm0, ymm0, ymm2; luego vextractf128 (carriles 2-3,
xmm0	Dejo la suma total	tras vaddpd+vaddpd, en double: sum_array la convierte a float
xmm1	Guardo n como float	vcvtsi2ss desde esi
xmm0	Cálculo la media	suma (ya en float) / n, se guarda en [rdx]

ETAPA 4 - Varianza: suma de (x - media)^2 acumulada en double		
Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8 (recalculado)
ymm0 / ymm4	Guardo dos acumuladores en double	ymm0 = elementos 0-3, ymm4 = elementos 4-7; sumas parciales
ymm1	Guardo la media en double, repetida en 4 carriles	vcvtss2sd desde [rdx], luego vbroadcastsd
ymm2 / ymm3	Guardo el bloque de trabajo en double	x convertido con vcvtps2pd, luego x-media, luego su
xmm2	Uno los dos acumuladores y guardo la mitad alta	vaddpd ymm0, ymm0, ymm4; luego vextractf128 (carriles 2-3,
xmm1	Guardo n convertido a double	vcvtsi2sd desde esi
xmm0	Dejo la varianza	suma de cuadrados / n en double (vdvsd)
xmm0	Guardo el resultado final	convertido a float (vcvtsd2ss) y guardado en [rcx]

ETAPA 5 - Mínimo y máximo con vmnpps / vmmpps (8 floats por vuelta)		
Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0	Guardo los mínimos parciales	8 carriles, inicializados con arr[0] (vbroadcastss)
ymm1	Guardo los máximos parciales	8 carriles, inicializados con arr[0] (vbroadcastss)
ymm2	Guardo el bloque actual	8 floats de arr
xmm2 / xmm3	Guardo temporales para reducir	mitad alta (vextractf128) y carriles intercambiados (vshufps)
xmm0	Dejo el mínimo final	carril 0, se guarda en [r8]
xmm1	Dejo el máximo final	carril 0, se guarda en [r9]

ETAPA 6 - normalize_array: (x - media) / desviación, 8 por vuelta + cola		
Registro	En palabras simples	Contenido concreto
rdi / rsi	Recibo entrada y salida	punteros in y out (alineados a 32 B)
edx	Recibo el tamaño	n
xmm0 / xmm1	Recibo la media y la desviación	mean y stddev (luego se pisan con el broadcast)
xmm8 / xmm9	Guardo copias de media y desviación	para comparar stddev con 0 y para la cola escalar
ymm0	Guardo la media en los 8 carriles	vbroadcastss desde xmm0
ymm1	Guardo la desviación en los 8 carriles	vbroadcastss desde xmm1
ymm2	Guardo el bloque de trabajo	in, luego in - media, luego dividido = out (vmovaps)
eax / r10d	Llevo el índice y el límite vectorial	i sube de 8 en 8; límite = n redondeado a múltiplo de 8
xmm3 / xmm2	Guardo el sobrante y su resultado (cola)	xmm3 = in[i]; xmm2 = (in[i] - media) / desviación

REFERENCIA - Versión escalar (SSE/SSE2): mismos roles, un solo elemento por vuelta		
Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 1, 2, ... (sube de 1 en 1 = 4 B)
xmm0	Guardo el acumulador en double	una sola suma en double (sum_array y varianza); en
xmm1	Guardo el elemento convertido a double	arr[i] via cvtsd2sd, o (arr[i]-media)/2 en double; en
xmm2	Guardo la media en double / el elemento a comparar	en varianza: cvtsd2sd desde [rbx], leída una vez y reutilizada,
xmm8 / xmm9	Guardo media y desviación	copias usadas en normalize_array (en float)
rbx, r12-r15	Respaldo de punteros	igual que en la vectorial; la media se lee de [rbx]

Cómo leer las tablas

En las etapas 3 y 4 (media y varianza), ymm se usa como 4 doubles de 64 bits (double, mayor precisión); en las etapas 5 y 6 (mínimo/máximo y normalización), ymm se usa como 8 floats de 32 bits. xmm es la mitad baja de un ymm, o un solo valor en código escalar. Callee-saved (hay que preservarlos): rbx, rbp, r12-r15. El resto son callee-saved.

Límite vectorial = n redondeado hacia abajo a múltiplo de 8; los n mod 8 sobrantes van a la cola escalar. vzoupper antes de cada ret limpia la mitad alta de los ymm.

Figura 4: Diagrama 3: asignación de registros por etapa

Figura 5: Diagrama 4: memoria y avance del puntero

